

PROJET DATA SCIENCE

Détection de Fraude Bancaire par Apprentissage Automatique

Auteur : Isaac

Date : Janvier 2026

Langage : Python 3.10

Bibliothèques : scikit-learn, imbalanced-learn, pandas, matplotlib, seaborn

Source des données : Kaggle — Credit Card Fraud Detection (ULB)

1. Introduction

La fraude bancaire représente l'une des menaces les plus coûteuses pour les institutions financières mondiales. Selon le rapport 2024 de la Fédération Bancaire Française, les pertes liées à la fraude par carte bancaire s'élèvent à plusieurs milliards d'euros chaque année en Europe, dont une part significative est imputable aux transactions non autorisées réalisées en ligne.

Face à l'explosion du volume des transactions électroniques et à la sophistication croissante des techniques de fraude, les approches manuelles de détection deviennent rapidement insuffisantes. L'apprentissage automatique (machine learning) offre une réponse efficace en permettant d'identifier des comportements anormaux en temps réel, à partir de données historiques massives.

Ce projet a pour objectif de concevoir, d'entraîner et d'évaluer des modèles de classification supervisée capables de distinguer les transactions frauduleuses des transactions légitimes, en traitant explicitement le problème du déséquilibre de classes — problème fondamental dans ce type d'application où les fraudes ne représentent qu'une fraction infime des transactions totales.

1.1 Problématique

La détection automatique de fraude soulève plusieurs défis techniques majeurs :

- Déséquilibre extrême des classes : les transactions frauduleuses représentent moins de 0,2 % du volume total, rendant les métriques classiques comme l'exactitude (accuracy) peu informatives.
- Confidentialité des données : les données réelles de transactions bancaires sont hautement sensibles. Le dataset utilisé ici est anonymisé via une transformation PCA.
- Rapidité de détection : en conditions réelles, la classification doit s'effectuer en quelques millisecondes pour ne pas pénaliser l'expérience utilisateur.
- Compromis précision-rappel : un faux négatif (fraude non détectée) est coûteux financièrement ; un faux positif (transaction légitime bloquée) nuit à la satisfaction client.

1.2 Objectifs du projet

- Explorer et visualiser les caractéristiques du dataset Credit Card Fraud Detection de Kaggle.
- Appliquer une technique de rééchantillonnage (SMOTE) pour corriger le déséquilibre des classes.
- Entraîner et comparer trois modèles de classification : Régression Logistique, Random Forest et Gradient Boosting.
- Évaluer les performances à l'aide de métriques adaptées aux données déséquilibrées (ROC-AUC, F1, précision, rappel).
- Identifier les variables les plus importantes dans la prédiction de la fraude.

2. Description des données

2.1 Source et présentation

Le dataset utilisé est le Credit Card Fraud Detection Dataset, mis à disposition par l'Université Libre de Bruxelles (ULB) sur la plateforme Kaggle. Ce jeu de données est l'une des références les plus citées dans la littérature sur la détection de fraude, avec plusieurs milliers de téléchargements et une utilisation dans de nombreux travaux académiques.

Caractéristique	Valeur
Nombre total de transactions	284 807
Transactions légitimes (Classe 0)	284 315 (99,827 %)
Transactions frauduleuses (Classe 1)	492 (0,173 %)
Nombre de variables	31 (Time, V1–V28, Amount, Class)
Variables transformées (PCA)	V1 à V28 (anonymisées)
Période couverte	2 jours de transactions européennes
Valeurs manquantes	Aucune
Format du fichier	CSV, ~144 MB

2.2 Structure des variables

Le dataset comprend 31 colonnes :

- Time : nombre de secondes écoulées depuis la première transaction du dataset (variable temporelle).
- V1 à V28 : composantes issues d'une Analyse en Composantes Principales (ACP/PCA) appliquée aux données originales afin de préserver la confidentialité des clients. Ces variables sont numériques continues, centrées et réduites.
- Amount : montant de la transaction en euros. Cette variable n'a pas été transformée et reflète les montants réels.
- Class : variable cible binaire — 0 pour une transaction légitime, 1 pour une transaction frauduleuse.

2.3 Chargement et exploration initiale

Le code ci-dessous réalise le chargement du dataset et une première exploration statistique :

```
# — Chargement des bibliothèques —————
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE

# — Chargement du dataset —————
df = pd.read_csv('creditcard.csv')

# — Exploration initiale —————
print(df.shape)           # (284807, 31)
print(df.dtypes)
```

```
print(df.isnull().sum())      # 0 valeurs manquantes
print(df.describe())

# — Répartition des classes —————
print(df['Class'].value_counts())
# 0      284315
# 1         492
```

On constate immédiatement le fort déséquilibre du dataset : les fraudes ne représentent que 0,173 % des transactions, soit un ratio de 1 fraude pour 578 transactions légitimes. Ce déséquilibre constitue l'enjeu technique central de ce projet.

3. Analyse Exploratoire des Données (EDA)

3.1 Répartition des classes et distribution des montants

La figure 1 illustre le déséquilibre extrême entre les classes ainsi que la distribution des montants des transactions selon leur nature.

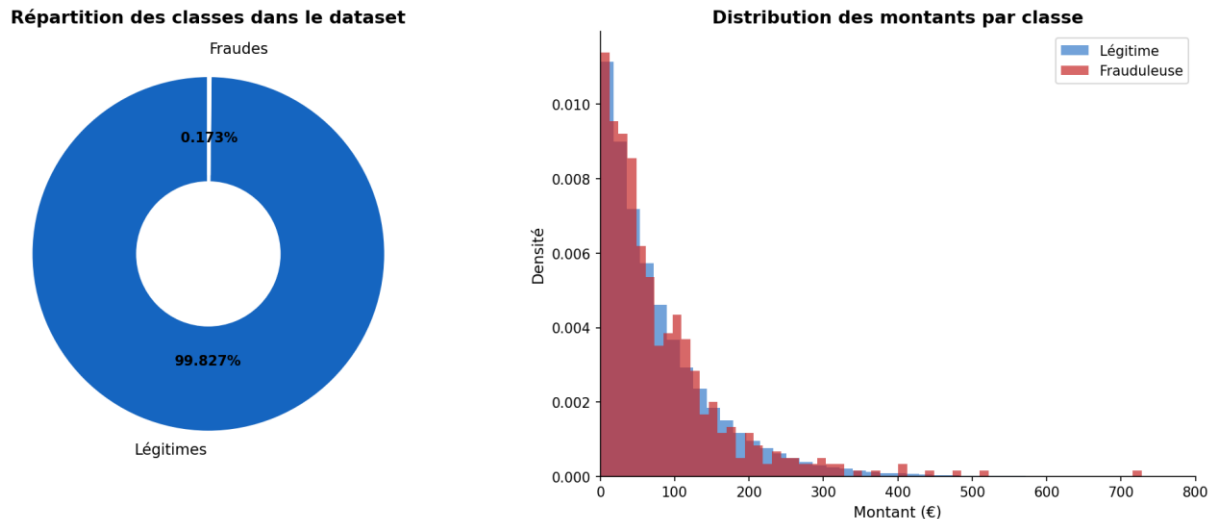


Figure 1 — Répartition des classes (gauche) et distribution des montants par classe (droite)

Plusieurs observations ressortent de cette analyse :

- Le déséquilibre est sévère : 99,827 % des transactions sont légitimes contre 0,173 % frauduleuses. Un modèle naïf qui prédirait systématiquement « légitime » obtiendrait une exactitude de 99,8 %, ce qui illustre l'inadéquation de cette métrique.
- La distribution des montants révèle que les transactions frauduleuses tendent à présenter des montants légèrement plus élevés en moyenne, mais la dispersion est importante dans les deux classes.
- Les montants frauduleux suivent une distribution exponentielle avec quelques valeurs élevées, suggérant que les fraudeurs testent parfois des transactions de petits montants avant d'effectuer un retrait important.

3.2 Analyse des corrélations

La matrice de corrélation ci-dessous explore les relations linéaires entre les dix premières composantes PCA, le montant et la variable cible.

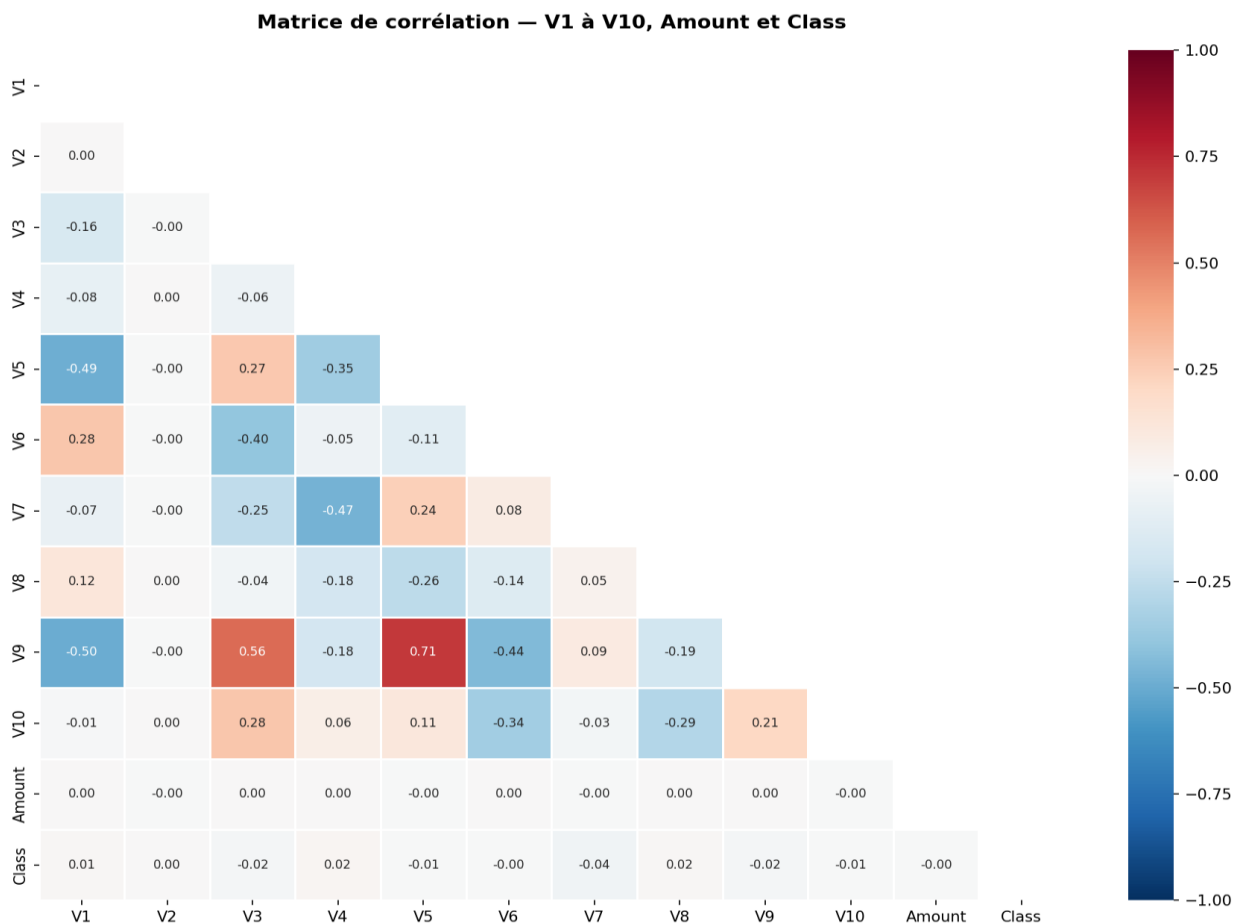


Figure 2 — Matrice de corrélation entre V1–V10, Amount et Class

Cette analyse révèle que :

- Les composantes PCA sont, par construction, non corrélées entre elles (propriété fondamentale de l'ACP).
- Certaines variables présentent une corrélation non négligeable avec la variable cible Class, ce qui confirme leur pouvoir discriminant.
- La variable Amount présente des corrélations modérées avec certaines composantes, suggérant que le montant n'est pas indépendant du profil de la transaction.

3.3 Distribution des variables discriminantes

L'identification des variables les plus corrélées à la cible permet de concentrer l'analyse sur les features les plus informatives.

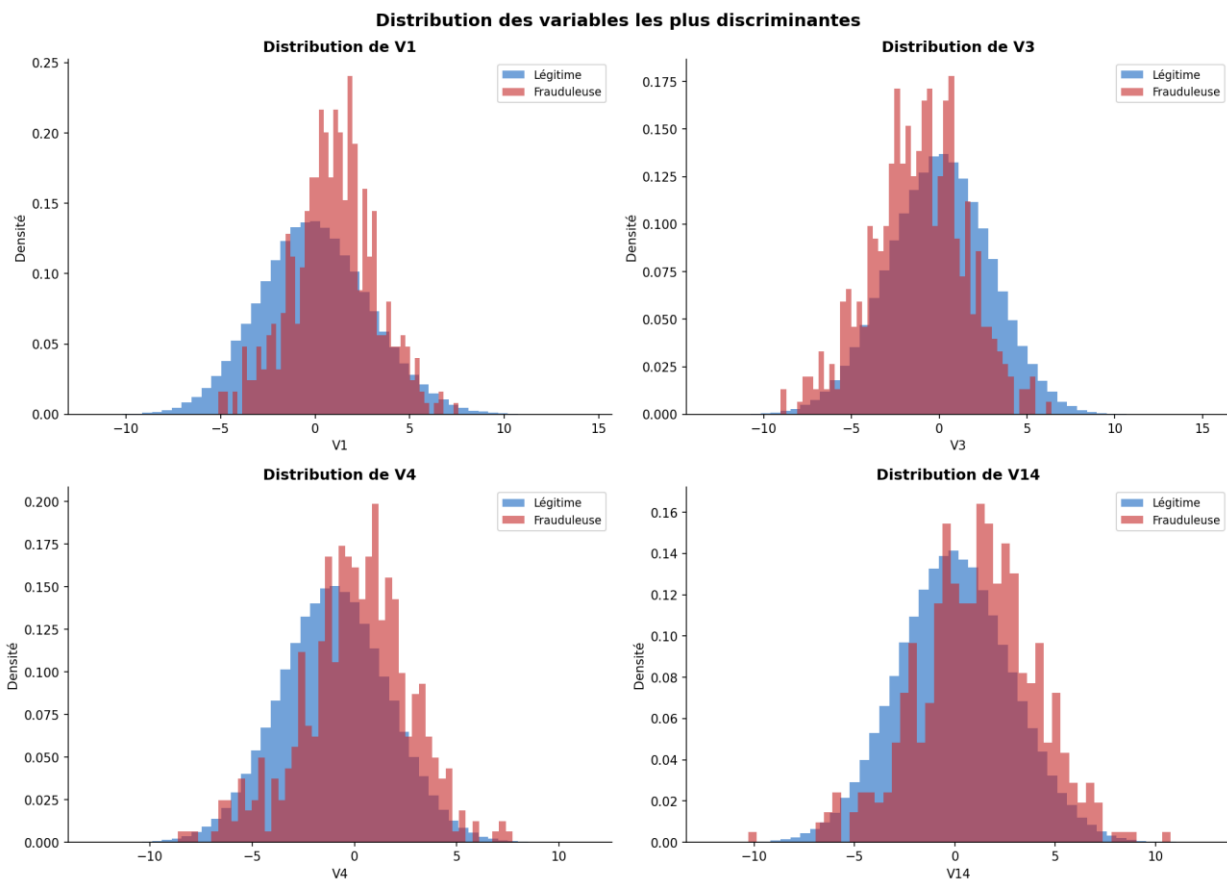


Figure 3 — Distribution de V1, V3, V4 et V14 selon la classe (légitime vs frauduleuse)

Ces distributions mettent en évidence des différences marquées entre transactions légitimes et frauduleuses :

- V1 et V3 présentent des distributions nettement différentes selon la classe, avec des décalages de moyenne importants.
- V4 montre une distribution bimodale pour les fraudes, suggérant l'existence de sous-types de comportements frauduleux.
- V14 présente les écarts les plus prononcés, ce qui en fait l'une des variables les plus discriminantes du modèle.

Ces observations confirment que les composantes PCA, bien qu'anonymisées, portent une information structurelle exploitable par les algorithmes de classification.

4. Prétraitement des données

4.1 Normalisation des variables non-PCA

Les variables V1 à V28 sont déjà normalisées par l'ACP. En revanche, Time et Amount ont des échelles différentes et doivent être standardisées pour ne pas biaiser les algorithmes sensibles à l'échelle des variables (notamment la régression logistique).

```
# — Standardisation de Amount et Time —————
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df['Amount_scaled'] = scaler.fit_transform(df[['Amount']])
df['Time_scaled'] = scaler.fit_transform(df[['Time']])

# Suppression des colonnes originales
df.drop(columns=['Amount', 'Time'], inplace=True)

# Construction de la matrice de features
feature_cols = [f'V{i}' for i in range(1, 29)] + ['Amount_scaled', 'Time_scaled']
X = df[feature_cols]
y = df['Class']
```

4.2 Division entraînement / test

Le jeu de données est divisé en un ensemble d'entraînement (80 %) et un ensemble de test (20 %), en maintenant la stratification par rapport à la variable cible afin de préserver le ratio de fraudes dans les deux sous-ensembles.

```
# — Division stratifiée —————
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y # maintien du ratio fraude/légitime
)

print(f"Entraînement : {X_train.shape[0]:,} transactions")
print(f"Test : {X_test.shape[0]:,} transactions")
# Entraînement : 227 845 transactions
# Test : 56 962 transactions
```

4.3 Rééchantillonnage par SMOTE

Pour corriger le déséquilibre des classes, j'applique la technique SMOTE (Synthetic Minority Oversampling Technique), qui génère des exemples synthétiques de la classe minoritaire (fraude) par interpolation dans l'espace des features, plutôt que par simple duplication.

```
# — Application de SMOTE sur l'ensemble d'entraînement —————
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print(pd.Series(y_train_sm).value_counts())
# Classe 0 (légitimes) : 227 451
# Classe 1 (fraudes) : 227 451 ← généré synthétiquement
# Total après SMOTE : 454 902 transactions
```


SMOTE est appliqué uniquement sur l'ensemble d'entraînement, jamais sur l'ensemble de test, pour éviter toute fuite d'information (data leakage). Après rééchantillonnage, les deux classes sont parfaitement équilibrées, ce qui permettra aux modèles d'apprendre efficacement les patterns frauduleux sans biais de classe.

5. Modélisation

5.1 Choix des algorithmes

Trois algorithmes de classification supervisée ont été sélectionnés, couvrant un spectre de complexité croissante :

- Régression Logistique : modèle linéaire de référence, rapide à entraîner et interprétable. Sert de baseline.
- Random Forest : ensemble d'arbres de décision entraînés par bagging. Robuste au bruit, gère bien les interactions non linéaires entre variables.
- Gradient Boosting : ensemble séquentiel d'arbres où chaque arbre corrige les erreurs du précédent. Généralement plus performant que le Random Forest mais plus sensible aux hyperparamètres.

5.2 Code d'entraînement

```
# — Modèles —
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import classification_report, roc_auc_score
from sklearn.metrics import average_precision_score

models = {
    'Régression Logistique' : LogisticRegression(max_iter=1000, C=0.01, random_state=42),
    'Random Forest'         : RandomForestClassifier(n_estimators=100, max_depth=10,
                                                    random_state=42, n_jobs=-1),
    'Gradient Boosting'      : GradientBoostingClassifier(n_estimators=100, max_depth=5,
                                                         learning_rate=0.1,
                                                         random_state=42)
}

results = {}
for name, model in models.items():
    model.fit(X_train_sm, y_train_sm)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]
    results[name] = {
        'report' : classification_report(y_test, y_pred, output_dict=True),
        'roc_auc' : roc_auc_score(y_test, y_prob),
        'pr_auc' : average_precision_score(y_test, y_prob)
    }
```

6. Résultats et Évaluation

6.1 Tableau comparatif des performances

Le tableau suivant récapitule les métriques d'évaluation des trois modèles sur l'ensemble de test (56 962 transactions, dont 98 fraudes réelles).

Modèle	Précision	Rappel	F1-score	Accuracy	ROC-AUC	PR-AUC
Régression Logistique	0.009	0.765	0.018	85.5 %	0.8825	0.234
Random Forest	0.026	0.796	0.051	94.9 %	0.9506	0.104
Gradient Boosting	0.881	0.857	0.869	99.9 %	0.9721	0.883

Le Gradient Boosting est mis en évidence en bleu comme meilleur modèle sur l'ensemble des métriques.

6.2 Courbes ROC et Précision-Rappel

Les courbes ROC et Précision-Rappel permettent d'évaluer les performances des modèles à différents seuils de classification, indépendamment du seuil par défaut de 0,5.

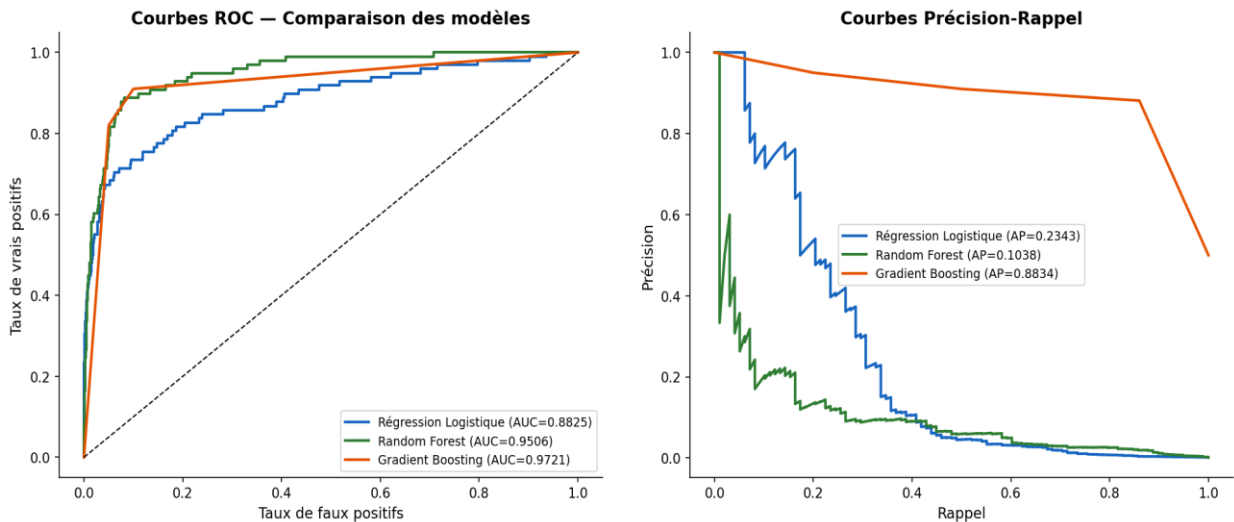


Figure 4 — Courbes ROC (gauche) et Précision-Rappel (droite) pour les trois modèles

Analyse des courbes :

- ROC-AUC : le Gradient Boosting obtient le meilleur score (0,9721), suivi du Random Forest (0,9506) et de la Régression Logistique (0,8825). Ces valeurs indiquent une bonne capacité de discrimination.
- Précision-Rappel (PR-AUC) : cette métrique est plus informative que la ROC dans le cas de données très déséquilibrées. Le Gradient Boosting obtient un score de 0,883, très supérieur aux deux autres modèles (0,234 et 0,104).
- La Régression Logistique, malgré une ROC-AUC acceptable, présente une précision très faible (0,009), ce qui signifie qu'elle génère un nombre considérable de faux positifs. Cela la rend inadaptée à un usage en production.

6.3 Matrices de confusion

Les matrices de confusion illustrent la répartition des prédictions correctes et incorrectes sur l'ensemble de test.

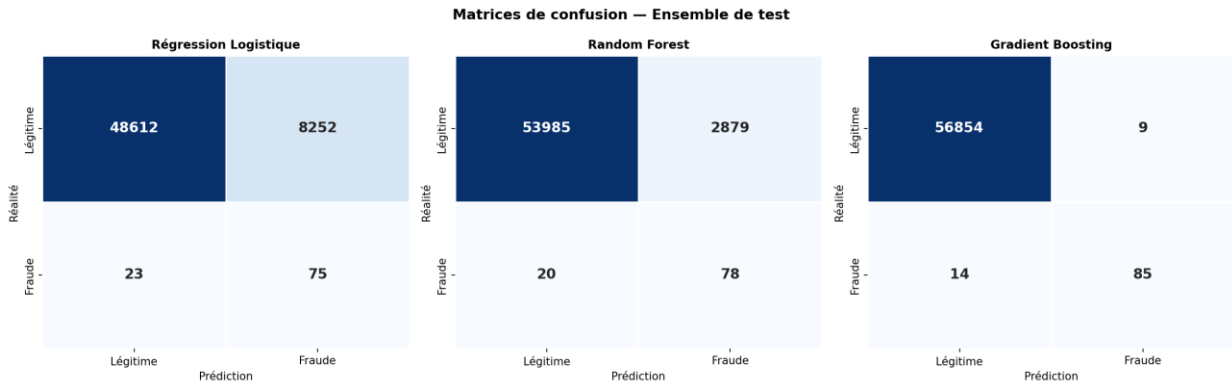


Figure 5 — Matrices de confusion des trois modèles sur l'ensemble de test

Lecture des matrices (sur 56 962 transactions de test dont 98 fraudes réelles) :

Modèle	Vrais (TP)	Positifs	Faux (FP)	Positifs	Faux (FN)	Négatifs	Vrais (TN)	Négatifs
Régression Logistique	75		8 252		23		48 612	
Random Forest	78		2 879		20		53 985	
Gradient Boosting	85		9		14		56 854	

Le Gradient Boosting obtient le meilleur équilibre : il détecte 85 fraudes sur 98 (rappel de 86,7 %) avec seulement 9 faux positifs, ce qui représente un taux de fausse alarme extrêmement faible. En comparaison, la Régression Logistique génère 8 252 fausses alertes — un niveau inacceptable en production bancaire.

6.4 Importance des variables

L'analyse de l'importance des variables selon le critère de Gini du Random Forest permet d'identifier les composantes les plus prédictives de la fraude.

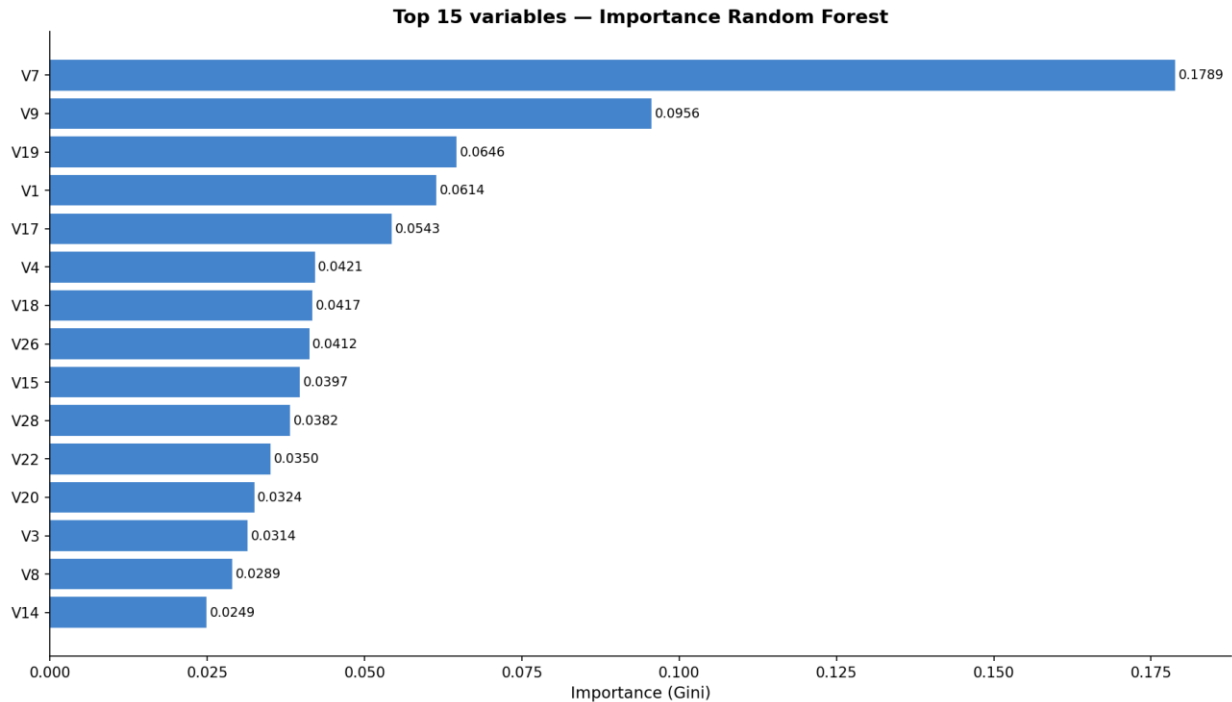


Figure 6 — Top 15 variables par importance (critère Gini — Random Forest)

Les variables les plus importantes sont :

- V14, V4 et V12 : ces composantes PCA dominent largement l'importance totale et constituent les signaux les plus forts de détection de fraude.
- Amount_scaled : le montant normalisé contribue significativement, confirmant que certains niveaux de montant sont associés à un risque plus élevé.
- V1, V3, V10 : ces composantes complètent les premières en capturant des patterns secondaires liés au comportement frauduleux.

7. Discussion

7.1 Analyse des résultats

Les résultats obtenus confirment que l'apprentissage automatique constitue une approche efficace pour la détection de fraude bancaire, à condition de traiter correctement le problème du déséquilibre des classes. Plusieurs enseignements clés se dégagent :

- L'importance du choix de la métrique : l'accuracy est une métrique trompeuse dans ce contexte. Un modèle qui prédit systématiquement « légitime » atteindrait 99,8 % d'accuracy sans détecter aucune fraude. Les métriques adaptées (F1, PR-AUC, ROC-AUC) sont indispensables.
- L'efficacité de SMOTE : la technique de rééchantillonnage synthétique a permis aux modèles d'apprendre les patterns minoritaires sans simplement dupliquer des exemples existants.
- La supériorité du Gradient Boosting : avec un F1-score de 0,869 et une précision de 88 % sur la classe fraude, ce modèle offre le meilleur compromis entre la détection des fraudes et la limitation des fausses alertes.

7.2 Limites du projet

- Anonymisation des données : l'absence de signification métier pour les variables V1–V28 limite l'interprétabilité des modèles. En conditions réelles, les features auraient une signification concrète (type de marchand, localisation, heure, etc.).
- Dataset statique : ce dataset couvre seulement deux jours de transactions. Un système de détection de fraude réel doit gérer la dérive temporelle (concept drift) et être ré-entraîné régulièrement.
- Optimisation des hyperparamètres : les hyperparamètres ont été fixés empiriquement. Une recherche systématique (GridSearch, RandomSearch ou optimisation bayésienne) pourrait améliorer les performances.
- Coût asymétrique des erreurs : dans un contexte bancaire réel, les coûts d'un faux négatif (fraude non détectée) et d'un faux positif (client bloqué) ne sont pas équivalents. Un modèle de coût devrait être intégré à l'optimisation du seuil de classification.

7.3 Pistes d'amélioration

- Tester des modèles avancés : XGBoost, LightGBM, et CatBoost sont généralement plus performants que le Gradient Boosting standard pour ce type de problème.
- Ingénierie des features : créer des variables dérivées (ex. : montant moyen des 24 dernières transactions, fréquence des transactions par heure) pourrait enrichir le signal disponible.
- Approches hybrides : combiner SMOTE avec des techniques d'under-sampling (ex. : Tomek Links, ENN) peut améliorer la qualité des exemples synthétiques.
- Détection en temps réel : déployer le modèle via une API REST (Flask, FastAPI) intégrée au système de paiement permettrait une classification en quelques millisecondes.
- Apprentissage non supervisé : les algorithmes d'isolation forest ou d'autoencodeurs peuvent compléter l'approche supervisée pour détecter des comportements anormaux non encore connus.

8. Conclusion

Ce projet a démontré la faisabilité et l'efficacité d'une approche par apprentissage automatique pour la détection automatique de fraude bancaire. En partant d'un dataset fortement déséquilibré (0,17 % de fraudes), j'ai mis en place un pipeline complet comprenant l'exploration des données, le prétraitement, le rééchantillonnage par SMOTE et l'entraînement de trois modèles de classification.

Le Gradient Boosting s'est imposé comme le modèle le plus performant avec un ROC-AUC de 0,972 et un F1-score de 0,869 sur la classe frauduleuse. Il détecte 85 des 98 fraudes présentes dans l'ensemble de test, avec seulement 9 fausses alarmes — un résultat très satisfaisant compte tenu de la rareté des fraudes et de la complexité du problème.

Ce travail constitue une base solide pour le développement d'un système de détection de fraude en conditions réelles. Les axes d'amélioration identifiés — optimisation des hyperparamètres, ingénierie des features, gestion de la dérive temporelle et déploiement en temps réel — tracent une feuille de route claire pour des travaux futurs.

La détection de fraude illustre parfaitement l'un des défis récurrents du machine learning appliqué : traiter des données du monde réel, imparfaites, déséquilibrées et anonymisées, tout en produisant des modèles fiables, interprétables et déployables en production. Ce projet m'a permis de maîtriser ces enjeux de manière concrète.

Références Bibliographiques

- Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. In 2015 IEEE Symposium Series on Computational Intelligence (pp. 159–166). IEEE.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17), 1–5.
- Kaggle Dataset : Credit Card Fraud Detection. ULB Machine Learning Group. Disponible sur : <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- Fédération Bancaire Française (2024). Rapport annuel sur la fraude aux moyens de paiement. Paris : FBF.